

Курсовая работа

Задача 1.3 — расчёт 101-й формы (*dm.dm_f101_round_f*) за январь 2018

Студент: Орисенко Максим

Дата: 09.05.2026

1. Постановка задачи

После того как в задаче 1.1 был наполнен детальный слой DS, а в задаче 1.2 на его основе рассчитаны витрины оборотов *dm.dm_account_turnover_f* и остатков *dm.dm_account_balance_f* за январь 2018, нам нужно собрать над этими витринами 101-ю форму — *dm.dm_f101_round_f*. Структура целевой витрины зафиксирована в «Структура таблиц.docx», расчёт ведётся по балансовым счетам второго порядка (первые пять символов *account_number*) и за весь отчётный период целиком (январь 2018).

По требованию задания нужно реализовать следующее:

- создать процедуру *dm.fill_f101_round_f(i_OnDate DATE)*, где *i_OnDate* — первый день месяца, следующего за отчётным; для расчёта за январь 2018 передаём 01.02.2018, отсюда отчётный период = 01.01.2018..31.01.2018, а «день, предшествующий первому дню отчётного периода» = 31.12.2017 (это и есть начальный остаток);
- в отчёт попадают все счета, действующие в любой день отчётного периода;
- группировка идёт по балансовым счетам второго порядка — *substr(account_number, 1, 5)*;
- *FROM_DATE* = первый день периода; *TO_DATE* = последний день периода;
- *CHAPTER* — глава из *ds.md_ledger_account_s*; *CHARACTERISTIC* — *char_type* из *ds.md_account_d*;
- *BALANCE_IN_RUB / BALANCE_IN_VAL / BALANCE_IN_TOTAL* — суммы *balance_out_rub* за день, предшествующий первому дню периода: *rub* только для рублёвых счетов (*currency_code* IN ('810','643')), *val* — для всех остальных, *total* — для всех;
- *TURN_DEB_RUB / TURN_DEB_VAL / TURN_DEB_TOTAL* — суммы *debit_amount_rub* за все дни периода с тем же разрезом по валюте;
- *TURN_CRE_RUB / TURN_CRE_VAL / TURN_CRE_TOTAL* — то же по *credit_amount_rub*;
- *BALANCE_OUT_RUB / BALANCE_OUT_VAL / BALANCE_OUT_TOTAL* — *balance_out_rub* за последний день периода;
- добавить логирование старта/окончания расчёта в *logs.etl_log*;
- перед расчётом удалять прежние записи по той же паре *from_date/to_date* — расчёт должен быть идемпотентным;
- рассчитать отчёт за январь 2018.

2. Стек и архитектура решения

Используется ровно тот же стек, что и в задачах 1.1/1.2: PostgreSQL 13 в Docker (контейнер `hw_coursework_postgres`, порт 5433 проброшен на хост, БД `airflow`, пользователь `airflow`, пароль `airflow`), Python 3 + `psycopg2` для запуска процедуры, DBeaver — для проверки результатов и логов. Сама логика расчёта 101 формы реализована на стороне Postgres в виде PL/pgSQL-процедуры `dm.fill_f101_round_f`. Витрины-источники (`dm.dm_account_turnover_f`, `dm.dm_account_balance_f`) уже наполнены в задаче 1.2.

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

NAMES	STATUS	PORTS
hw_coursework_postgres	Up 56 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> |
```

Рис. 1. Контейнер `hw_coursework_postgres` запущен, проброшен порт `5433 → 5432`.

В DBeaver видны три рабочие схемы — `ds` (детальный слой из 1.1), `logs` (общий журнал ETL) и `dm` (целевые витрины), куда записывается результат:

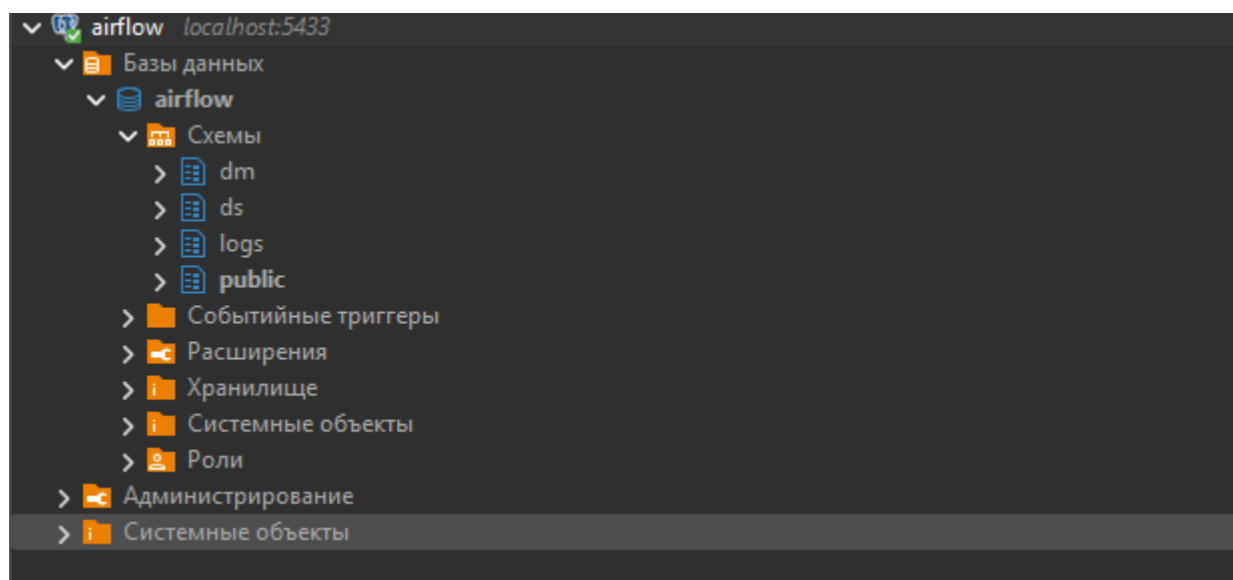


Рис. 2. Дерево схем БД `airflow` в DBeaver: `dm`, `ds`, `logs`, `public`.

3. Структура проекта (фокус на 1.3)

`hw_coursework_task1/`

├─ `sql/`

| └─ `01_ddl.sql` схемы `ds`, `logs`, `dm` + таблицы (включая `dm.dm_f101_round_f`)

| └─ `02_procedures.sql` все процедуры: `fill_account_turnover_f` и

| `fill_account_balance_f` (1.2) + `fill_f101_round_f` (1.3)

└─ `python/`

└─ `db.py` параметры подключения

└─ `run_f101.py` запуск процедуры за январь 2018 (`i_OnDate=2018-02-01`)

4. DDL целевой витрины (sql/01_ddl.sql)

Витрина 101-й формы определена ровно по «Структура таблиц.docx»: пять идентифицирующих полей (from_date / to_date / chapter / ledger_account / characteristic) и двенадцать числовых полей в трёх группах по четыре — входящие остатки, дебет/кредит обороты и исходящие остатки, каждая в разрезе rub / val / total. Все числовые поля имеют тип NUMERIC(23,8) — этого достаточно для рублёвого пересчёта с копейками.

```
CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f
(
    from_date      DATE,
    to_date        DATE,
    chapter        CHAR(1),
    ledger_account  CHAR(5),
    characteristic CHAR(1),
    balance_in_rub  NUMERIC(23,8),
    balance_in_val  NUMERIC(23,8),
    balance_in_total NUMERIC(23,8),
    turn_deb_rub    NUMERIC(23,8),
    turn_deb_val    NUMERIC(23,8),
    turn_deb_total  NUMERIC(23,8),
    turn_cre_rub    NUMERIC(23,8),
    turn_cre_val    NUMERIC(23,8),
    turn_cre_total  NUMERIC(23,8),
    balance_out_rub NUMERIC(23,8),
    balance_out_val NUMERIC(23,8),
    balance_out_total NUMERIC(23,8)
);
```

Рис. 3. Фрагмент 01_ddl.sql со структурой dm.dm_f101_round_f — 17 полей.

Полная DM-секция DDL (включая таблицы из задачи 1.2 и страховочную копию dm.dm_f101_round_f_v2 для импорта в 1.4) приведена ниже:

```
CREATE TABLE IF NOT EXISTS dm.dm_account_turnover_f (
```

```
on_date DATE,
```

```
account_rk BIGINT,
```

```
credit_amount NUMERIC(23,8),
```

```
credit_amount_rub NUMERIC(23,8),
```

```
debet_amount NUMERIC(23,8),
```

```
debet_amount_rub NUMERIC(23,8)
```

```
);
```

```
CREATE TABLE IF NOT EXISTS dm.dm_account_balance_f (
```

```
on_date DATE,
```

```

account_rk BIGINT,
balance_out NUMERIC(23,8),
balance_out_rub NUMERIC(23,8)
);
CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f (
from_date DATE,
to_date DATE,
chapter CHAR(1),
ledger_account CHAR(5),
characteristic CHAR(1),
balance_in_rub NUMERIC(23,8),
balance_in_val NUMERIC(23,8),
balance_in_total NUMERIC(23,8),
turn_deb_rub NUMERIC(23,8),
turn_deb_val NUMERIC(23,8),
turn_deb_total NUMERIC(23,8),
turn_cre_rub NUMERIC(23,8),
turn_cre_val NUMERIC(23,8),
turn_cre_total NUMERIC(23,8),
balance_out_rub NUMERIC(23,8),
balance_out_val NUMERIC(23,8),
balance_out_total NUMERIC(23,8)
);

```

-- Копия для проверки импорта в задании 1.4.

```

CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f_v2 (LIKE dm.dm_f101_round_f INCLUDING
ALL);

```

5. Процедура расчёта (dm.fill_f101_round_f)

Входной параметр `i_OnDate` — первый день месяца, следующего за отчётным. Внутри процедуры из него получаются три ключевые даты:

- `v_from_date = i_OnDate - 1 month` — первый день отчётного периода;

- `v_to_date = i_OnDate - 1 day` — последний день отчётного периода;
- `v_prev_date = v_from_date - 1 day` — день, предшествующий периоду (для входящих остатков).

Логика процедуры:

- логируем событие START в `logs.etl_log` с полем `extra=period=v_from_date..v_to_date`;
- идемпотентный DELETE строк по паре (`from_date`, `to_date`) — расчёт можно перезапускать многократно;
- CTE `accounts_period` — все уникальные счета, действующие в любой день отчётного периода (пересечение интервала действия счёта в `md_account_d` с [`v_from_date`; `v_to_date`]); сразу извлекаем `ledger_account = substr(account_number, 1, 5)`, `char_type` и `currency_code`;
- CTE `chapters` — справочник «балансовый счёт второго порядка → глава» из `md_ledger_account_s` (`MAX(chapter)` на случай если у одного счёта в справочнике несколько строк по разным датам);
- CTE `bal_in / bal_out` — остатки (`balance_out_rub`) за `v_prev_date` и `v_to_date` соответственно из `dm.dm_account_balance_f`;
- CTE `turn_period` — суммы дебет/кредит-оборотов в рублях по каждому счёту за весь период из `dm.dm_account_turnover_f`;
- CTE `enriched` — ширина = `accounts_period` LEFT JOIN со всеми справочниками и обороткой; COALESCE отсутствующих остатков/оборотов в 0 (счёт без проводок всё равно попадает в отчёт);
- финальный INSERT с GROUP BY `ledger_account`, в котором каждое число агрегируется тремя выражениями SUM с CASE: `rub` — `currency_code` IN ('810','643'), `val` — `currency_code` NOT IN ('810','643'), `total` — без условия;
- GET DIAGNOSTICS `v_rows = ROW_COUNT` и лог END с `rows_affected`;
- блок EXCEPTION WHEN OTHERS — пишем событие ERROR с текстом `SQLERRM` и пробрасываем исключение наружу.

```

143 -- 1.3 Расчёт 101 формы.
144 -- i_OnDate – первый день месяца, следующего за отчётным.
145 CREATE OR REPLACE PROCEDURE dm.fill_f101_round_f(i_OnDate DATE)
146 LANGUAGE plpgsql AS $$
147 DECLARE
148     v_started    TIMESTAMP := clock_timestamp();
149     v_from_date  DATE := (i_OnDate - INTERVAL '1 month')::date;
150     v_to_date    DATE := (i_OnDate - INTERVAL '1 day')::date;
151     v_prev_date  DATE := v_from_date - INTERVAL '1 day';
152     v_rows       BIGINT;
153 BEGIN
154     INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
155     VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'START',
156             v_started, format('period=%s..%s', v_from_date, v_to_date));
157
158     DELETE FROM dm.dm_f101_round_f
159     WHERE from_date = v_from_date AND to_date = v_to_date;
160
161     WITH accounts_period AS (
162         -- счета, действующие в любой день отчётного периода
163         SELECT DISTINCT
164             a.account_rk,
165             substr(a.account_number,1,5) AS ledger_account,
166             a.char_type,
167             a.currency_code
168         FROM ds.md_account_d a
169         WHERE a.data_actual_date <= v_to_date
170               AND a.data_actual_end_date >= v_from_date
171     ),
172     chapters AS (
173         SELECT ledger_account::text AS ledger_account, MAX(chapter) AS chapter
174         FROM ds.md_ledger_account_s
175         GROUP BY ledger_account
176     ),
177     bal_in AS (
178         SELECT account_rk, balance_out_rub
179         FROM dm.dm_account_balance_f
180         WHERE on_date = v_prev_date
181     ),
182     bal_out AS (
183         SELECT account_rk, balance_out_rub
184         FROM dm.dm_account_balance_f
185         WHERE on_date = v_to_date
186     ),
187     turn_period AS (
188         SELECT account_rk,
189             SUM(debet_amount_rub) AS turn_deb_rub_total,
190             SUM(credit amount rub) AS turn_cre_rub_total

```

Рис. 4. fill_f101_round_f, часть 1: DECLARE с расчётом v_from_date/v_to_date/v_prev_date, лог START, идемпотентный DELETE, CTE accounts_period / chapters / bal_in / bal_out.

```

186 ),
187 turn_period AS (
188     SELECT account_rk,
189            SUM(debet_amount_rub) AS turn_deb_rub_total,
190            SUM(credit_amount_rub) AS turn_cre_rub_total
191     FROM dm.dm_account_turnover_f
192     WHERE on_date BETWEEN v_from_date AND v_to_date
193     GROUP BY account_rk
194 ),
195 enriched AS (
196     SELECT a.ledger_account,
197            c.chapter,
198            a.char_type,
199            a.currency_code,
200            COALESCE(bi.balance_out_rub, 0) AS bal_in_rub,
201            COALESCE(bo.balance_out_rub, 0) AS bal_out_rub,
202            COALESCE(tp.turn_deb_rub_total, 0) AS turn_deb_rub_total,
203            COALESCE(tp.turn_cre_rub_total, 0) AS turn_cre_rub_total
204     FROM accounts_period a
205     LEFT JOIN chapters c ON c.ledger_account = a.ledger_account
206     LEFT JOIN bal_in bi ON bi.account_rk = a.account_rk
207     LEFT JOIN bal_out bo ON bo.account_rk = a.account_rk
208     LEFT JOIN turn_period tp ON tp.account_rk = a.account_rk
209 )
210 INSERT INTO dm.dm_f101_round_f(
211     from_date, to_date, chapter, ledger_account, characteristic,
212     balance_in_rub, balance_in_val, balance_in_total,
213     turn_deb_rub, turn_deb_val, turn_deb_total,
214     turn_cre_rub, turn_cre_val, turn_cre_total,
215     balance_out_rub, balance_out_val, balance_out_total)
216 SELECT v_from_date, v_to_date,
217        MAX(chapter), ledger_account, MAX(char_type),
218        SUM(CASE WHEN currency_code IN ('810','643') THEN bal_in_rub ELSE 0 END),
219        SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_in_rub ELSE 0 END),
220        SUM(bal_in_rub),
221        SUM(CASE WHEN currency_code IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),
222        SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),
223        SUM(turn_deb_rub_total),
224        SUM(CASE WHEN currency_code IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),
225        SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),
226        SUM(turn_cre_rub_total),
227        SUM(CASE WHEN currency_code IN ('810','643') THEN bal_out_rub ELSE 0 END),
228        SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_out_rub ELSE 0 END),
229        SUM(bal_out_rub)
230 FROM enriched
231 GROUP BY ledger_account;
232

```

Рис. 5. *fill_f101_round_f*, часть 2: CTE *turn_period* и *enriched*, финальный INSERT с GROUP BY *ledger_account* и тремя SUM(CASE) на каждое числовое поле — разрез rub / val / total.

```

230      FROM enriched
231      GROUP BY ledger_account;
232
233      GET DIAGNOSTICS v_rows = ROW_COUNT;
234
235      INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)
236      VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'END',
237             v_started, clock_timestamp(), v_rows,
238             format('period=%s..%s', v_from_date, v_to_date));
239  EXCEPTION WHEN OTHERS THEN
240      INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)
241      VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'ERROR',
242             v_started, clock_timestamp(), SQLERRM);
243      RAISE;
244  END;
245  $$;
246

```

Рис. 6. *fill_f101_round_f*, часть 3: *GET DIAGNOSTICS*, лог *END* с *rows_affected*, *EXCEPTION WHEN OTHERS* с записью события *ERROR* и *RAISE*.

5.1. Полный текст процедуры

-- i_OnDate — первый день месяца, следующего за отчётным.

```
CREATE OR REPLACE PROCEDURE dm.fill_f101_round_f(i_OnDate DATE)
```

```
LANGUAGE plpgsql AS $$
```

```
DECLARE
```

```
v_started TIMESTAMP := clock_timestamp();
```

```
v_from_date DATE := (i_OnDate - INTERVAL '1 month')::date;
```

```
v_to_date DATE := (i_OnDate - INTERVAL '1 day')::date;
```

```
v_prev_date DATE := v_from_date - INTERVAL '1 day';
```

```
v_rows BIGINT;
```

```
BEGIN
```

```
INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
```

```
VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'START',
```

```
v_started, format('period=%s..%s', v_from_date, v_to_date));
```

```
DELETE FROM dm.dm_f101_round_f
```

```
WHERE from_date = v_from_date AND to_date = v_to_date;
```

```
WITH accounts_period AS (
```

```
-- счета, действующие в любой день отчётного периода
```

```
SELECT DISTINCT
```

```

a.account_rk,
substr(a.account_number,1,5) AS ledger_account,
a.char_type,
a.currency_code
FROM ds.md_account_d a
WHERE a.data_actual_date <= v_to_date
AND a.data_actual_end_date >= v_from_date
),
chapters AS (
SELECT ledger_account::text AS ledger_account, MAX(chapter) AS chapter
FROM ds.md_ledger_account_s
GROUP BY ledger_account
),
bal_in AS (
SELECT account_rk, balance_out_rub
FROM dm.dm_account_balance_f
WHERE on_date = v_prev_date
),
bal_out AS (
SELECT account_rk, balance_out_rub
FROM dm.dm_account_balance_f
WHERE on_date = v_to_date
),
turn_period AS (
SELECT account_rk,
SUM(debet_amount_rub) AS turn_deb_rub_total,
SUM(credit_amount_rub) AS turn_cre_rub_total
FROM dm.dm_account_turnover_f
WHERE on_date BETWEEN v_from_date AND v_to_date

```

```

GROUP BY account_rk

),

enriched AS (

SELECT a.ledger_account,

c.chapter,

a.char_type,

a.currency_code,

COALESCE(bi.balance_out_rub, 0) AS bal_in_rub,

COALESCE(bo.balance_out_rub, 0) AS bal_out_rub,

COALESCE(tp.turn_deb_rub_total, 0) AS turn_deb_rub_total,

COALESCE(tp.turn_cre_rub_total, 0) AS turn_cre_rub_total

FROM accounts_period a

LEFT JOIN chapters c ON c.ledger_account = a.ledger_account

LEFT JOIN bal_in bi ON bi.account_rk = a.account_rk

LEFT JOIN bal_out bo ON bo.account_rk = a.account_rk

LEFT JOIN turn_period tp ON tp.account_rk = a.account_rk

)

INSERT INTO dm.dm_f101_round_f(

from_date, to_date, chapter, ledger_account, characteristic,

balance_in_rub, balance_in_val, balance_in_total,

turn_deb_rub, turn_deb_val, turn_deb_total,

turn_cre_rub, turn_cre_val, turn_cre_total,

balance_out_rub, balance_out_val, balance_out_total)

SELECT v_from_date, v_to_date,

MAX(chapter), ledger_account, MAX(char_type),

SUM(CASE WHEN currency_code IN ('810','643') THEN bal_in_rub ELSE 0 END),

SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_in_rub ELSE 0 END),

SUM(bal_in_rub),

SUM(CASE WHEN currency_code IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),

```

```

SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),
SUM(turn_deb_rub_total),
SUM(CASE WHEN currency_code IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),
SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),
SUM(turn_cre_rub_total),
SUM(CASE WHEN currency_code IN ('810','643') THEN bal_out_rub ELSE 0 END),
SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_out_rub ELSE 0 END),
SUM(bal_out_rub)
FROM enriched
GROUP BY ledger_account;
GET DIAGNOSTICS v_rows = ROW_COUNT;
INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)
VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'END',
v_started, clock_timestamp(), v_rows,
format('period=%s..%s', v_from_date, v_to_date));
EXCEPTION WHEN OTHERS THEN
INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)
VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'ERROR',
v_started, clock_timestamp(), SQLERRM);
RAISE;
END;
$$;

```

6. Чистый старт перед демонстрацией

Чтобы наглядно показать, что данные в витрину 101-й формы приходят именно из нашей процедуры, перед запуском очищаем витрину и удаляем предыдущие лог-события того же процесса:

```

TRUNCATE TABLE dm.dm_f101_round_f;
DELETE FROM logs.etl_log WHERE process = 'fill_f101_round_f';

```

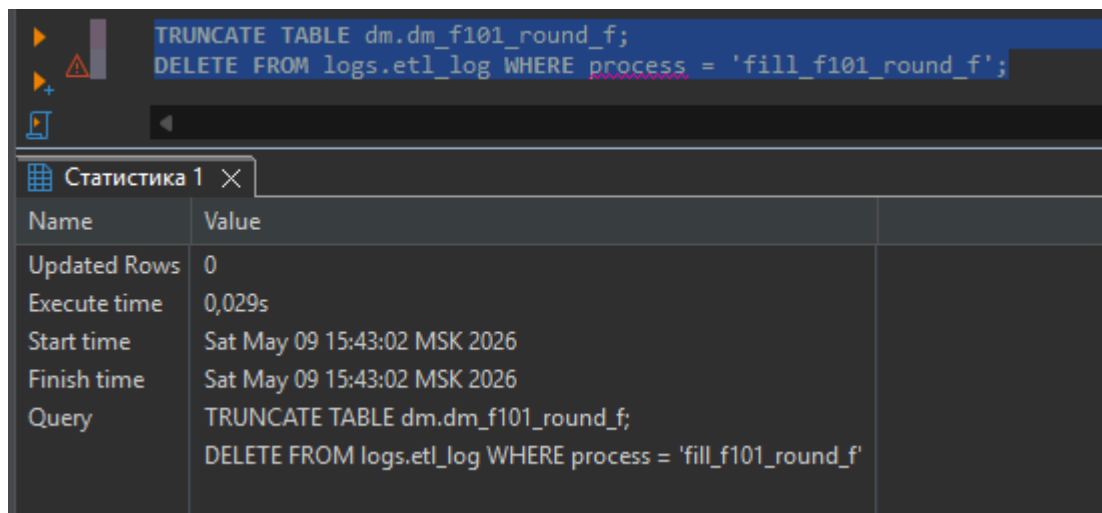


Рис. 7. TRUNCATE + DELETE логов в DBeaver. Updated Rows = 0 — БД уже находилась в чистом состоянии после предварительной подготовки демо, обе команды отработали идемпотентно (как и должны).

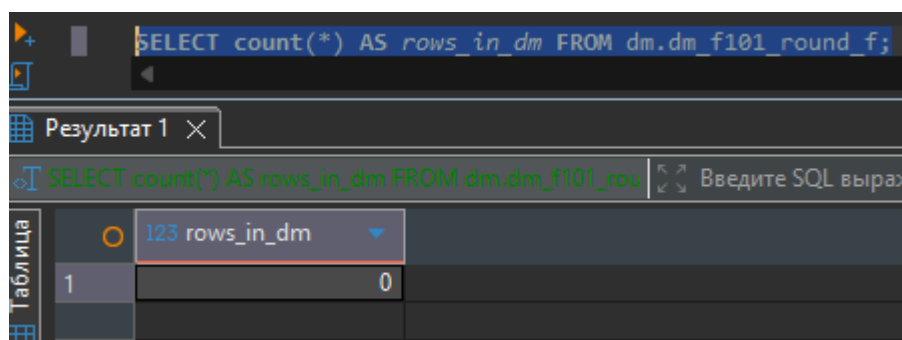


Рис. 8. Контрольный запрос — витрина пуста: rows_in_dm = 0.

7. Запуск расчёта за январь 2018

Скрипт run_f101.py получает соединение через общий db.py и одной командой CALL dm.fill_f101_round_f(2018-02-01) запускает расчёт. Сразу после CALL он делает SELECT count(*) FROM dm.dm_f101_round_f и печатает число строк, попавших в витрину. Если процедура отработает с ошибкой — psusorg2 откатит транзакцию и пробросит исключение наружу.

```

1  from __future__ import annotations
2
3  from datetime import date
4
5  import psycopg2
6
7  from db import DB_CONFIG
8
9
10 def main() -> None:
11     on_date = date(2018, 2, 1)
12     with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
13         cur.execute("CALL dm.fill_f101_round_f(%s);", (on_date,))
14         cur.execute("SELECT count(*) FROM dm.dm_f101_round_f;")
15         n = cur.fetchone()[0]
16         print(f"[OK] dm.dm_f101_round_f - строк: {n}")
17
18
19 if __name__ == "__main__":
20     main()
21

```

Рис. 9. Код run_f101.py — CALL dm.fill_f101_round_f и контрольный SELECT count(*).

"""Запуск процедуры расчёта 101 формы за январь 2018 (i_OnDate=2018-02-01)."""

from __future__ import annotations

from datetime import date

import psycopg2

from db import DB_CONFIG

def main() → None:

on_date = date(2018, 2, 1)

with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:

cur.execute("CALL dm.fill_f101_round_f(%s);", (on_date,))

cur.execute("SELECT count(*) FROM dm.dm_f101_round_f;")

n = cur.fetchone()[0]

print(f"[OK] dm.dm_f101_round_f - строк: {n}")

if __name__ == "__main__":

main()

Запуск из терминала:

```
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> python run_f101.py
[OK] dm.dm_f101_round_f - строк: 18
```

Рис. 10. `python run_f101.py` → `[OK] dm.dm_f101_round_f` - строк: 18.

8. Результаты расчёта в `dm.dm_f101_round_f`

В витрину легло 18 строк — это ровно число уникальных балансовых счетов второго порядка, действующих в январе 2018 в нашем датасете. У всех 18 строк `from_date` = 2018-01-01, `to_date` = 2018-01-31 и `chapter` = «А» (раздел «Капитал и резервы / Межбанковские операции» — все наши счета имеют `ledger_account` вида 301xx, 302xx или 304xx).

	date	to_date	AZ chapter	AZ ledger_account	AZ characteristic	123 balance_in_total	123 turn_deb_total	123 turn_cre_total	123 balance_out_total
1	2018-01-01	2018-01-31	A	30102	A	200 079 650,12	625 440 427,12	787 848 410,3	37 671 666,94
2	2018-01-01	2018-01-31	A	30109	П	63 891,96	0	0	63 891,96
3	2018-01-01	2018-01-31	A	30110	A	16 518 754,724378	373 332,34	1 083 668,65	15 808 418,414378
4	2018-01-01	2018-01-31	A	30111	П	125 006 707,42	262 884 223,89	172 294 943,56	34 417 427,09
5	2018-01-01	2018-01-31	A	30114	A	1 460 552 792,87861871	1 464 972 651,18680107	2 199 251 369,68591323	726 274 074,37950655
6	2018-01-01	2018-01-31	A	30126	П	67 224,06	0	0	67 224,06
7	2018-01-01	2018-01-31	A	30202	A	129 554	0	93 819,58	35 734,42
8	2018-01-01	2018-01-31	A	30204	A	5 541,83	149 647,04	0	155 188,87
9	2018-01-01	2018-01-31	A	30220	П	1 429 793 492,98439384	251 131 748,618938	162 558 540,257008	1 341 220 284,62246384
10	2018-01-01	2018-01-31	A	30221	A	21 517 297,872542	0	0	21 517 297,872542
11	2018-01-01	2018-01-31	A	30222	П	43 584 074,79796	5 595 511,517496	0	37 988 563,280464
12	2018-01-01	2018-01-31	A	30223	П	302 098,75	6 349 676,77	15 781 776,55	9 734 198,53
13	2018-01-01	2018-01-31	A	30232	П	4 604 117,728464	0	0	4 604 117,728464
14	2018-01-01	2018-01-31	A	30233	A	5 860 414,611544	0	0	5 860 414,611544
15	2018-01-01	2018-01-31	A	30236	П	7 191 673,831935	134 845 152,115113	249 074 167,076237	121 420 688,793059
16	2018-01-01	2018-01-31	A	30413	A	185 530,94	2 098 780,35	2 382 487,14	-98 175,85
17	2018-01-01	2018-01-31	A	30424	A	7 855 306,53	4 414 156,66	9 299 232,48	2 970 230,71
18	2018-01-01	2018-01-31	A	30425	A	2 294 752,421626	0	0	2 294 752,421626

Рис. 11. `dm.dm_f101_round_f` — все 18 строк (`from_date`, `to_date`, `chapter`, `ledger_account`, `characteristic`, `balance_in_total`, `turn_deb_total`, `turn_cre_total`, `balance_out_total`).

Развёрнутая разбивка для самой нагруженной строки — `ledger_account` = 30114 (корреспондентские счета в банках-нерезидентах, ведутся в иностранной валюте). Видно, что вся сумма приходит в столбцах `val` (для счетов с `currency_code` не из {810, 643}) и одновременно равна `total` — а `*_rub` нулевые: рублёвых счетов в этом `ledger`'е в датасете нет.

	AZ ledger_account	AZ characteristic	123 balance_in_rub	123 balance_in_val	123 balance_in_total	123 turn_deb_rub	123 turn_deb_val	123 turn_deb_total	123
1	30114	A	0	1 460 552 792,87861871	1 460 552 792,87861871	0	1 464 972 651,18680107	1 464 972 651,18680107	123

Рис. 12. Разбивка `ledger=30114`, левая часть: `balance_in_rub=0`, `balance_in_val=balance_in_total=1 460 552 792.88 руб.`, `turn_deb_total=1 464 972 651.19 руб.`

SQL query: `SELECT ledger_account, characteristic, balance_in_rub, balance_in_val, balance_in_total, turn_deb_rub, turn_deb_val, turn_deb_total, turn_cre_rub, turn_cre_val, turn_cre_total, balance_out_rub, balance_out_val, balance_out_total FROM dm.dm_f101_round_f WHERE ledger_account = '30114';`

	123 turn_deb_val	123 turn_deb_total	123 turn_cre_rub	123 turn_cre_val	123 turn_cre_total	123 balance_out_rub	123 balance_out_val	123 balance_out_total
1	0	1 464 972 651,18680107	1 464 972 651,18680107	0	2 199 251 369,68591323	2 199 251 369,68591323	0	726 274 074,37950655

Рис. 13. Разбивка ledger=30114, правая часть: turn_deb_val=turn_deb_total, turn_cre_rub=0, turn_cre_val=turn_cre_total=2 199 251 369.69 руб., balance_out_val=balance_out_total=726 274 074.38 руб.

9. Логирование процедуры

Логирование ведётся в общую таблицу logs.etl_log из задачи 1.1 — process='fill_f101_round_f', object_name='dm.dm_f101_round_f'. На каждый вызов процедуры в логе появляются ровно две записи: START (без finished_at) и END (с finished_at и количеством вставленных строк). В поле extra и у START, и у END дублируется отчётный период period=v_from_date..v_to_date.

SQL query: `SELECT log_id, process, event, started_at, finished_at, rows_affected, extra FROM logs.etl_log WHERE process = 'fill_f101_round_f' ORDER BY log_id;`

	123 log_id	AZ process	AZ event	started_at	finished_at	123 rows_affected	AZ extra
1	405	fill_f101_round_f	START	2026-05-09 12:44:03.738	[NULL]	[NULL]	period=2018-01-01..2018-01-31
2	406	fill_f101_round_f	END	2026-05-09 12:44:03.738	2026-05-09 12:44:03.764	18	period=2018-01-01..2018-01-31

Рис. 14. После одного запуска процедуры — пара log_id 405 (START) / 406 (END), rows_affected=18, extra=period=2018-01-01..2018-01-31.

10. Демонстрация идемпотентности

ТЗ требует возможности перезапускать расчёт за один и тот же отчётный период многократно — поэтому в начале процедуры стоит DELETE по паре (from_date, to_date). Демонстрация: считаем число строк в витрине, повторно запускаем CALL для той же даты, считаем строки после.

SQL query: `SELECT count(*) AS rows_before FROM dm.dm_f101_round_f;`

	123 rows_before
1	18

Рис. 15. До повторного запуска: rows_before = 18.

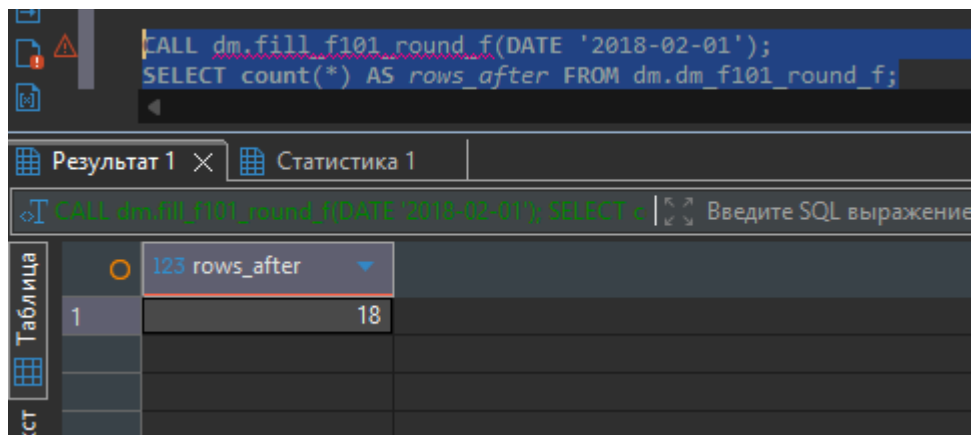


Рис. 16. CALL dm.fill_f101_round_f(DATE '2018-02-01') и сразу SELECT count(*) — rows_after = 18. Дублей нет, DELETE+INSERT отработали корректно.

А вот в логе появляется новая пара START/END — то есть видно ровно два независимых запуска одной и той же даты:

log_id	event	started_at	finished_at	rows_affected	extra
405	START	2026-05-09 12:44:03.738	[NULL]	[NULL]	period=2018-01-01..2018-01-31
406	END	2026-05-09 12:44:03.738	2026-05-09 12:44:03.764	18	period=2018-01-01..2018-01-31
407	START	2026-05-09 15:45:42.067	[NULL]	[NULL]	period=2018-01-01..2018-01-31
408	END	2026-05-09 15:45:42.067	2026-05-09 15:45:42.082	18	period=2018-01-01..2018-01-31

Рис. 17. logs.etl_log по process=fill_f101_round_f — две пары START/END: log_id 405–406 (первый запуск) и 407–408 (повторный).

11. Контрольная проверка инварианта total = rub + val

По логике витрины для каждого числового поля выполняется тождество total = rub + val (любой счёт либо рублёвый, либо нет). Запрос-агрегат проверяет это сразу для четырёх показателей — balance_in, turn_deb, turn_cre, balance_out — на всех 18 строках.

SELECT

SUM(CASE WHEN balance_in_total <> COALESCE(balance_in_rub,0) + COALESCE(balance_in_val,0) THEN 1 ELSE 0 END) AS bad_in,

SUM(CASE WHEN turn_deb_total <> COALESCE(turn_deb_rub,0) + COALESCE(turn_deb_val,0) THEN 1 ELSE 0 END) AS bad_deb,

SUM(CASE WHEN turn_cre_total <> COALESCE(turn_cre_rub,0) + COALESCE(turn_cre_val,0) THEN 1 ELSE 0 END) AS bad_cre,

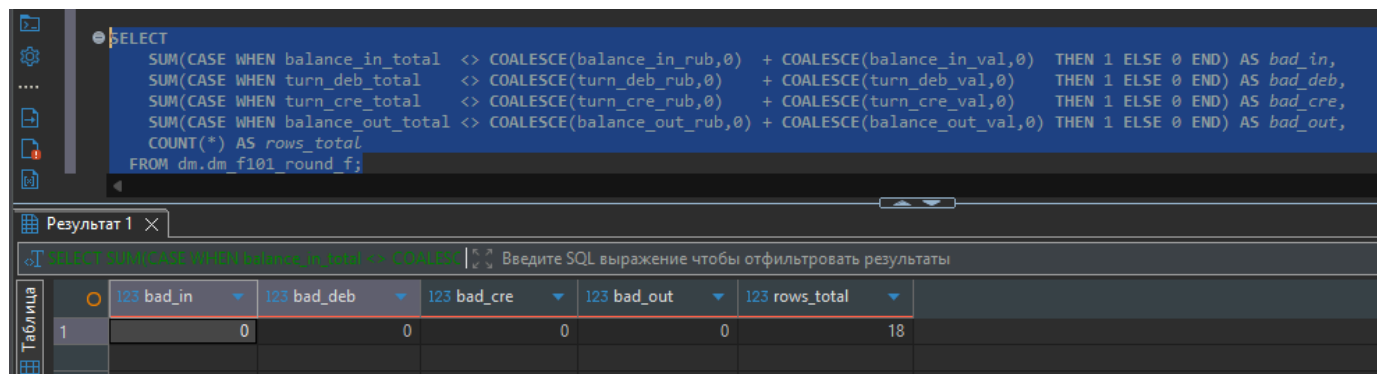
```

SUM(CASE WHEN balance_out_total <> COALESCE(balance_out_rub,0) +
COALESCE(balance_out_val,0) THEN 1 ELSE 0 END) AS bad_out,

COUNT(*) AS rows_total

FROM dm.dm_f101_round_f;

```



Результат 1

	123 bad_in	123 bad_deb	123 bad_cre	123 bad_out	123 rows_total
1	0	0	0	0	18

Рис. 18. Контроль: $bad_in=0$, $bad_deb=0$, $bad_cre=0$, $bad_out=0$ при $rows_total=18$ — инвариант $total = rub + val$ выполняется на всех строках по всем четырём показателям.

12. Итог по задаче 1.3

- создана и применена процедура `dm.fill_f101_round_f(i_OnDate DATE)`, точно соответствующая ТЗ: входной параметр — первый день месяца, следующего за отчётным; внутри пересчитывается в `[v_from_date; v_to_date]` и `v_prev_date`;
- группировка идёт по балансовым счетам второго порядка (`substr(account_number, 1, 5)`);
- `rub / val / total` строятся одной агрегацией с тремя `SUM(CASE)` — `rub`: `currency_code IN ('810','643')`; `val`: `NOT IN (...)`; `total`: без условия;
- предусмотрена идемпотентность — `DELETE WHERE from_date=v_from_date AND to_date=v_to_date` в начале процедуры;
- логирование `START/END/ERROR` ведётся в единую таблицу `logs.etl_log` из 1.1; в `extra` записывается отчётный период;
- расчёт за январь 2018 выполнен — в `dm.dm_f101_round_f` легло 18 строк по балансовым счетам второго порядка, все из главы «А»;
- идемпотентность подтверждена реальным демо: при повторном `CALL` число строк не изменилось ($18 \rightarrow 18$), а в логе появилась новая пара `START/END`;
- инвариант $total = rub + val$ выполняется на всех 18 строках по всем четырём показателям (`balance_in / turn_deb / turn_cre / balance_out`).

Этим закрываются все требования задачи 1.3 курсовой работы.

13. Ссылки

Репозиторий GitHub (ветка 1_3): https://github.com/kodi842/neoflex_coursework/tree/1_3

Видео-демонстрация (Google Drive, все 4 видео по курсовой):
<https://drive.google.com/drive/folders/1ZMhXnrIE3nR8zUqTHC2uECeeShwDaIFG?usp=sharing>